Assignment 3
Due by 11:59 pm on 8 November 2025

Covered contents:
Lectures 7-9

Submission method:
Follow the submission instructions given at the end of each question. **Failure to comply will lead to deduction of 5 points from your grade for this assignment.**

A note on using generative tools:
You are welcome to use generative tools to assist you in completing assignments. However, you are not allowed to simply copy and paste; you should understand the outputs and submit works that are completed based on your own understanding. Failure to do so may result in penalties.

All coding questions must be done in Python.

If your have any questions, please email phycsan@ust.hk.

Question 1 [20 points]

In this problem, your goal is to implement a parallel histogram function similar to that in Assignment 2 by using `mpi4py`. Since by nature different processes do no share address spaces, we cannot use a global `results` array like we did with the shared-memory version. Instead, all communication must be done explicitly. Here's one possible implementation:

- A master process (say process 0) scatters data to all processes
- Each process computes their own local histogram
- The master process gathers and combines all local histograms into the final historam

Submission instruction:

Submit your work as `"histogram_mpi.py"`. A skeleton code with the same file name is included along with this assignment.

Question 2 [10 points]

We are given an $n \times d$ `numpy` array of Cartesian coordinates of n points in d-dimension. Our goal is to compute the $n \times n$ distance matrix D defined by

$$D_{ij} = \text{Euclidean distance between point } i \text{ and point } j. \tag{1}$$

(a) Compute the distance matrix by using nested `for` loops.

(b) You might notice that the nested loops in (a) are very slow for large n and d. Compute D by using only `numpy` functions. Do not use any for loops. You should notice that your vectorized version runs much faster.

Submission instruction:

Submit your work as `"distance.py"`. A skeleton code with the same file name is included along with this assignment.

Question 3 [30 points]

Laplace's equation reads $\nabla^2\phi = 0$. It is an important equation governing a lot of systems in physics. For example, imagine having an enclosed space, and the temperature distribution at the boundary is specified. In this case, the temperature distribution in the interior is given by the solution to Laplace's equation with the specified boundary condition.

For this question, let's assume that we are working in a two-dimensional (flat) space, which is a square of size 1 by 1 (in some unit). In this space, Laplace's equation becomes

$$\frac{\partial^2\phi}{\partial x^2} + \frac{\partial^2\phi}{\partial y^2} = 0. \tag{2}$$

One method to numerically solve (2) is to replace the space with its discretized version: Let's discretize by turning the square into an $(n+1) \times (n+1)$ grid of points, which we label by $\phi_{i,j}$. See fig. 1 below.

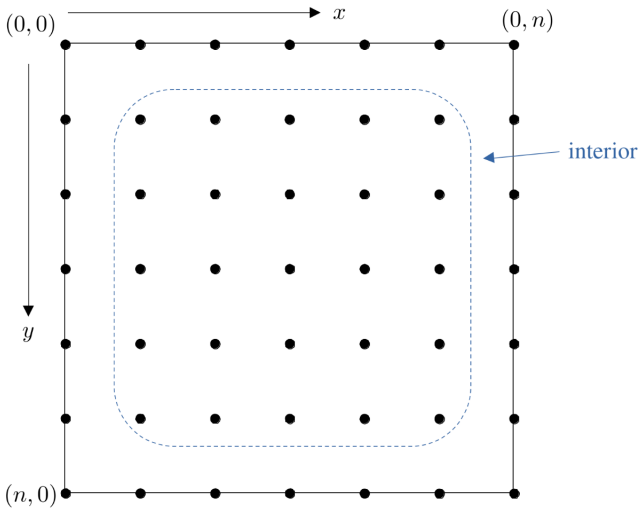

Figure 1: Discretization by a square grid of points.

The spacing of the grid is given by $h = 1/n$.

(a) Under this discretization scheme, show that the second derivative at the interior is approximately given by

$$\left.\frac{\partial^2\phi}{\partial x^2}\right|_{(i,j)} \approx \frac{\phi_{i-1,j} - 2\phi_{i,j} + \phi_{i+1,j}}{h^2}. \tag{3}$$

Derive a similar expression for the y-derivative.

(b) Using your results in (a), show that the Laplace equation can be approximately replaced by the following set of equations (one each for each pair of (i, j)):

$$\phi_{i,j} = \frac{1}{4}(\phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1}, \phi_{i,j-1}), \tag{4}$$

where $1 \leq i < n$ and $1 \leq j < n$.

One can devise an iterative scheme to numerically solve eq. (4)[1]:

```
def update(phi_new, phi_old):
    m, n = phi_new.shape

    for r in range(1, m-1):
        for c in range(1, n-1):
            phi_new[r, c] = 0.25 * (phi_old[r+1, c] + phi_old[r-1, c] + \
                                    phi_old[r, c+1] + phi_old[r, c-1])

phi_old = initialize() # see part (c) below
max_diff = np.inf
iter = 0

while (iter <= iter_max) and (max_diff >= tol):
    phi_new = initialize() # see part (c) below
    update(phi_new, phi_old)

    max_diff = np.max(np.abs(phi_new - phi_old))
    phi_old = phi_new
    iter += 1
```

In the algorithm above, `iter_max` is the maximum number of iterations allowed, `tol` is the error tolerance below which we define the algorithm to have converged. (We haven't proved the algorithm will actually converge to the correct answer, but let's just assume this is the case.)

(c) Write the `initialize` function which accepts five inputs:

- `phi_top`: value of ϕ at the top boundary
- `phi_bottom`: value of ϕ at the bottom boundary
- `phi_left`: value of ϕ at the left boundary
- `phi_right`: value of ϕ at the right boundary
- `n`: size of the grid minus 1

This function should return an $(n+1)\times(n+1)$ two-dimensional `numpy` array which satisfies the boundary conditions and has value zero for all interior points.

(d) Inside the `update` function, there are explicit uses of Python for loops which, as we have emphasized during lecture, are very slow. Your task is to

- Rewrite `update` to eliminate all explicit for loops.
  You are only allowed to use functions from the numpy library.

(e) For some reason, you are still not happy with the speed of your `numpy` version. Your task is to

- Rewrite the entire algorithm in Cython (you can use loops now).

How does the performance compare with your cleverly vectorized `numpy` version? Is this what you expected?

(f) Suppose the boundary conditions are

$$\phi_{\text{top}} = \phi_{\text{botoom}} = 100,\ \phi_{\text{left}} = \phi_{\text{right}} = 50.$$

Use one of your implementations above to numerically solve the Laplace equation with $n = 100$, `iter_max` $= 10000$, and `tol` $= 10^{-5}$. Submit your solution as a 2D `numpy` array saved as a `.npy` file by using the `np.save()` function.

[1] Why don't we just directly solve the system of linear equations?

The next question (part (g)) is optional. It does not count towards your grades. Do not submit any code files for this part.

(g) Actually, the code I gave you above is really inefficient! Are you able to spot where the inefficiency is? (Profiling might help...) If so, go back and improve your codes, and see how much faster your code runs now!

Submission instructions:

For parts (a) and (b), submit your work as `"question_4.pdf"`

For parts (c) and (d), submit your work as `"laplace_solver.py"`

For part (e), submit your work as `"laplace_solver_cython.pyx"`, along with the corresponding `set_up.py` file.

For part (f), submit your work as `"laplace_solution.npy"`

Skeleton codes with the same file names are included along with this assignment.